

## Content

| <b>Sr. No.</b> | <b>Title</b>                     | <b>Page No.</b> |
|----------------|----------------------------------|-----------------|
| 1              | Abstract                         | 2               |
| 2              | Introduction                     | 4               |
| 3              | Problem Statement                | 6               |
| 5              | Objectives                       | 7               |
| 5              | Literature Review / Related Work | 8               |
| 6              | Dataset / Input Description      | 9               |
| 7              | Methodology / System Design      | 10              |
| 8              | Requirement Analysis             | 12              |
| 9              | Implementation Summary           | 13              |
| 10             | Evaluation Methodology           | 14              |
| 11             | Workflow Diagrams                | 15              |
| 12             | Screenshot / Results             | 17              |
| 13             | Future Implementation            | 21              |
| 14             | Bibliography                     | 22              |
| 15             | Conclusion                       | 23              |

## Abstract

In an increasingly competitive global job market, the resume acts as a crucial document that represents a candidate's professional identity. However, the effectiveness of a resume is not only dependent on the applicant's skills or experience but also on its structure, language, and compliance with Applicant Tracking Systems (ATS) used by recruiters and enterprises. Over 75% of resumes are rejected before reaching human eyes due to non-compliance with ATS protocols or lack of keyword optimization. As such, creating an optimized, well-structured, and keyword-aligned resume becomes a significant challenge, particularly for students and entry-level professionals with limited exposure to recruitment mechanisms.

My implementation combined a document parser (for extracting sections and entities from PDF/DOCX), a scoring module (heuristics and measurements for ATS compatibility), an LLM-driven recommendation engine (for content rephrasing and keyword optimization), a resume-templating engine (HTML → PDF conversion with CSS tuned for print), and an interactive visualization layer to present evaluation metrics. I built the platform using Python and Streamlit, integrated NLP toolkits and TF-IDF-based similarity, and wrapped an LLM (Gemini) behind a utility layer for controlled prompts and responses.

To address this critical need, we present R.E.A.D.M.L. (Resume Enhancing, Analysing & Development using Machine Learning) — an AI-powered, end-to-end system that assists users in building impactful resumes or analysing existing ones using a combination of natural language processing (NLP), machine learning scoring metrics, AI-generated feedback, and real-time visual analytics. The system caters to two distinct use cases: (1) guided resume creation, and (2) intelligent resume analysis. In the first mode, the user is led through an intuitive form-based interface to enter their educational, professional, and personal details, which are dynamically converted into a clean, professional HTML template and exported to PDF using wkhtmltopdf. In the second mode, users can upload an existing resume in PDF or DOCX format, which is parsed using regular expressions and NLP techniques. The content is analysed for structure, formatting, readability, skill alignment, and keyword density, after which the system assigns an ATS score across five key dimensions.

## Introduction

In the contemporary digital landscape, where the hiring process is increasingly driven by automation and artificial intelligence, a resume has evolved from being just a document to a data-driven artifact that determines the first impression of a candidate in front of an employer. In fact, studies have shown that nearly 75% of resumes are rejected by Applicant Tracking Systems (ATS) before they ever reach a human recruiter. Despite this widespread reality, many job seekers, especially students and entry-level professionals, lack awareness about how to tailor their resumes for both human readability and machine parsing. This gap in understanding often becomes a barrier between skillful candidates and potential job opportunities.

Traditional resume builders, whether web-based platforms or offline tools, typically focus on visual aesthetics or template formatting. However, they often overlook the deeper layers of content optimization such as keyword alignment, semantic relevance to job descriptions, grammatical coherence, and ATS-friendliness. Furthermore, these platforms usually do not provide intelligent feedback, making it difficult for users to understand the flaws in their resumes or the scope for improvement. As a result, candidates continue to circulate resumes that may look professional but fail to perform under automated evaluation systems.

To tackle this challenge, we developed R.E.A.D.M.L. – Resume Enhancing, Analysing & Development using Machine Learning, a next-generation, AI-powered resume assistant built using Python, Streamlit, Natural Language Processing (NLP), and the Gemini API. Unlike traditional resume builders, R.E.A.D.M.L. goes beyond formatting. It brings intelligence into the resume creation and evaluation process by enabling real-time resume generation, automated content analysis, ATS scoring, and context-aware AI feedback. It empowers users to either build a new resume from scratch using a guided form interface or upload an existing resume to receive an in-depth analysis with actionable suggestions.

The system operates through two primary workflows: Resume Generation and Resume Analysis. In the Resume Generation workflow, users provide structured input through a dynamic form that captures all essential resume sections such as personal information, education, projects, skills, and achievements. The system then converts this data into a professional HTML format and renders it as a downloadable PDF using the wkhtmltopdf engine. This ensures that even candidates with no design experience can generate clean, well structured resumes that meet industry standards.

One of the most powerful features of R.E.A.D.M.L. is its integration with the Gemini API, a large language model (LLM) developed by Google. After analysing the resume, the system sends the parsed data to the Gemini model, which then provides intelligent feedback including improvements to the summary section, suggestions for missing or weak skills, and personalized rewrite prompts based on the target job description. This feedback loop transforms a generic resume into a role-specific, optimized document, significantly improving the candidate's chances of passing ATS filters and catching recruiter attention.

The development of R.E.A.D.M.L. was driven by a clear objective: to bridge the gap between job seekers and intelligent recruitment systems. In doing so, it contributes not only to the success of individual candidates but also to educational institutions, training programs, and career support systems that aim to make their students and clients more employable in the AI era. The system is designed to be extensible and adaptable, with a clear roadmap for future enhancements such as cover letter generation, LinkedIn profile parsing, mobile compatibility, and cloud-based resume hosting.

In summary, R.E.A.D.M.L. is not just a resume builder; it is a complete resume intelligence platform. It brings together the power of machine learning, natural language processing, and user-centric design to offer a unique solution to one of the most common but overlooked problems in the recruitment pipeline. By helping candidates craft smarter, more effective resumes, R.E.A.D.M.L. ensures that their skills don't go unnoticed — and their potential isn't lost in translation.

## Problem Statement

I tackled the following problems:

- Many resumes were rejected by ATS systems for reasons unrelated to candidate ability (poor parsing, unconventional formatting, embedded images).
- Job seekers often did not know which keywords or phrasing would improve ATS ranking.
- Manual resume reviews by counsellors were time-consuming and subjective.

My goal was to build an explainable, repeatable pipeline that extracted structured information from varied resume formats, measured ATS compatibility along interpretable axes, and generated context-aware rewrite suggestions that preserved candidate voice.

## Objectives

I set these objectives and achieved them:

1. I provided a form-driven resume builder that produced polished HTML and PDF resumes.
2. I implemented a parser to extract structured fields from uploaded PDFs/DOCX (personal details, profile, education, skills, projects, experience, achievements).
3. I designed an interpretable multi-dimensional ATS scoring mechanism: formatting, content coverage, skills match, keyword density, and readability.
4. I integrated an LLM recommendation engine to produce role-specific rewrite suggestions that prioritized ATS keywords while maintaining readability.
5. I built interactive visualizations to present component scores and suggestions, enabling users to accept or reject changes.
6. I enforced privacy-conscious handling of API keys and temporary files and documented reproducible deployment steps.

## Literature Review / Related Work

I reviewed the existing literature and products to guide my design:

1. **ATS Mechanisms:** I studied vendor whitepapers and research showing that ATS extract standard headings and keywords; plain-text structured resumes parsed more reliably than design-heavy PDFs.
2. **Resume Parsing Techniques:** I evaluated rule-based, machine-learning, and hybrid parsers. I chose a hybrid approach using deterministic heuristics combined with spaCy-based entity extraction to balance explainability and robustness.
3. **Keyword Matching:** I applied TF-IDF and cosine similarity as baseline techniques for keyword matching and considered embedding-based semantic similarity for future enhancements.
4. **LLM-Assisted Rewriting:** I investigated prompt-engineering patterns that produced targeted rewrites LMs can rephrase content to match a role's expected tokens while retaining meaning.
5. **Evaluation Practices:** I adopted industry recommendations: combine automatic metrics (parsing precision/recall, similarity scores) with human judgments from career counsellors to measure practical effectiveness.

I cited spaCy, TF-IDF literature, Streamlit docs, and LLM provider documentation in my references.

## Dataset / Input Description

R.E.A.D.M.L. consumed structured form inputs and uploaded resume documents rather than a static dataset. The inputs I handled were:

1. **Form inputs:** User-provided structured fields (name, contact, profile summary, education entries, skills list, projects and responsibilities). I normalized these into a structured `resume_data` JSON and used it for templating.
2. **Uploaded documents:**
  - **PDFs:** I extracted text while addressing issues like hyphenation, headers/footers, and page breaks.
  - **DOCX:** I extracted paragraphs and runs to preserve structural cues.
3. **Test corpus:** For evaluation, I curated an anonymized test set containing:
  - Entry-level student resumes,
  - Mid-level resumes,
  - A variety of formats (single-column, two-column, and graphically dense layouts).
4. **Job descriptions:** I used job descriptions as reference text for keyword matching and similarity scoring.

In the report I describe the number of test documents, representative anonymized examples, and any input cleaning steps I applied (whitespace normalization, hyphen-merge, header/footer stripping).



## Methodology / System Design

### High-level architecture

I designed R.E.A.D.M.L. as a set of modular components:

1. **Frontend (Streamlit)** — Two workflows: Create Resume and Analyze Resume. The UI handled session state, form inputs, file uploads, and visualization rendering.
2. **Resume Templating Engine** — I transformed structured `resume_data` into an A4-optimized HTML template, ensuring page-break-friendly CSS for reliable PDF export.
3. **Resume Parser** — I implemented a multi-stage pipeline:
  - **Document type detection:** PDF vs DOCX.
  - **Text extraction:** For PDFs, I applied heuristics to rejoin split lines and handle hyphenation; for DOCX, I used paragraph runs.
  - **Section segmentation:** I matched headings against a curated list (case-insensitive, trimmed), used positional heuristics, and applied fallback rules when headings were not explicit.
  - **Entity extraction:** I used spaCy NER plus domain-specific pattern matching to extract institutions, degrees, dates, companies, roles, and skill tokens.
  - **Post-processing:** I normalized dates, canonicalized skill tokens, deduplicated items, and grouped responsibilities.
4. **ATS Scoring Module** — I implemented component scores:
  - **Format Score:** presence and parseability of standard sections, and absence of blocking formatting elements.
  - **Content Score:** descriptive richness in experience and projects (presence of action verbs, quantified achievements).
  - **Skills Score:** normalized skill coverage against a canonical skill set.
  - **Keyword Score:** similarity between resume tokens and job-description tokens (TF-IDF similarity and keyword density).
  - **Readability Score:** sentence-length and Flesch–Kincaid-like heuristics.

I combined these into a weighted Total ATS Score; the report documents the weightings I used and the rationale behind them.

5. **LLM Recommendation Engine** — I wrapped the LLM with controlled prompts that:
  - Asked for concise rewrite suggestions that increase keyword alignment,
  - Requested suggested edits to bullet points to include quantified impact where possible,
  - Returned brief justifications for each suggestion.

I enforced privacy by sending only necessary excerpts and by post-processing outputs to avoid hallucinated facts.

6. **Visualization & UX** — I used interactive charts (radar, gauge, donut) to surface component scores, and provided inline accept/reject toggles for suggested edits.
7. **PDF Generation** — I converted the HTML template to a printable PDF, validated size/page-count, and offered the generated file for download.

## **Workflow details**

### **Resume Creation (Form → PDF)**

- I validated form inputs, normalized dates, and limited bullet counts for layout stability.
- I generated HTML using the templating engine and converted it to PDF for download.

### **Resume Analysis (Uploaded doc → Scores + Suggestions)**

- I parsed the uploaded document into structured data and normalized tokens.
- I computed component scores and the Total ATS Score.
- I queried the LLM with a concise context and obtained rewrite suggestions prioritized by expected ATS impact.
- Users could preview suggested edits and accept selected ones; accepted edits were regenerated into a new PDF.

# Requirement Analysis

## • Software Requirements

| Component                  | Description                                                           |
|----------------------------|-----------------------------------------------------------------------|
| Operating System           | Windows 10/11, macOS, or any Linux-based system                       |
| Programming Language       | Python 3.10+                                                          |
| Python Environment Manager | venv, virtualenv, or conda (recommended for dependency isolation)     |
| Package Manager            | pip or uv (for installing Python libraries)                           |
| Frontend Framework         | Streamlit – Used to build the web interface                           |
| AI/ML API Integration      | Gemini AP I– For AI-powered resume suggestions                        |
| PDF Rendering Engine       | Wkhtmltopdf – Converts HTML to PDF                                    |
| Data Visualization         | Plotly – For rendering graphs and charts                              |
| Resume Parsing             | Custom NLP module using re, python-docx, and PyPDF2 libraries         |
| Web Browser                | Google Chrome / Firefox – For launching and viewing the Streamlit app |
| Text Editor / IDE          | VS Code / PyCharm / Jupyter Lab – For development and debugging       |
| API Secret Management      | .streamlit/secrets.toml – Stores Gemini API keys securely             |

I built the system using:

- **Python** (3.8+ recommended) — primary implementation language.
- **Streamlit** — for rapid UI development and session management.
- **spaCy** — for tokenization, POS tagging, and NER.
- **scikit-learn (TF-IDF)** — for keyword extraction and similarity scoring.
- **PyPDF2 / python-docx** — for document text extraction.
- **wkhtmltopdf / pdf conversion tool** — for reliable HTML→PDF conversion and print-quality output.
- **Plotly** — for interactive visualizations integrated in Streamlit.
- **LLM (Gemini)** — used for content rewriting; I wrapped the calls to control prompts and costs.
- **pytest** — for unit tests and mocking LLM responses.

I recorded the exact versions used in `requirements.txt` and noted environment-specific dependencies (e.g., wkhtmltopdf system package).

## Implementation Summary

I describe the behavior, inputs, outputs, edge cases, and failure modes for each module (no code).

### Frontend (Streamlit)

I built a user-friendly interface with form validation, session-managed state, and clear error messages. I implemented features to:

- Validate required fields and date inputs,
- Provide warnings for unsupported file types,
- Show progress and error dialogs for LLM calls and PDF generation,
- Allow users to accept or reject suggested edits.

### Parser

I focused on robustness:

- I used a combination of heading-matching heuristics and spaCy NER to identify sections and entities,
- I normalized skills to a canonical list and merged synonyms,
- I handled hyphenation and split lines in PDFs,
- I flagged image-only PDFs as requiring OCR and recommended adding OCR in future work.

I documented known failure modes, such as two-column tables or infographic resumes that reduce parsing accuracy.

### Scoring & Recommendations

- I computed component scores with transparent criteria, documented the thresholds, and justified weightings.
- I limited the number of suggestions per resume to avoid overwhelming users, and prioritized suggestions by expected score improvement.

### LLM Interactions

- I provided only minimal context to the LLM to minimize PII exposure,
- I used bounded prompts to reduce hallucination,
- I required user confirmation before applying any LLM-produced text to an official resume.

### PDF Generator

- I designed CSS to avoid page-break-inside issues and tested on common printers.
- I validated output PDF properties and provided the user with a downloadable file and a temporary storage policy.

## Evaluation Methodology

I established a rigorous evaluation plan to validate parser accuracy and suggestion effectiveness.

### Metrics

- **Parsing Accuracy:** Section detection precision/recall and entity extraction accuracy measured against labeled ground truth.
- **Scoring Validity:** Correlation between ATS scores and expert human judgments.
- **Suggestion Effectiveness:** Pre/post ATS-score delta, recruiter ratings, and downstream simulation of ATS pass rates.
- **Usability:** Task completion time, suggestion accept rate, and user satisfaction scores.

### Experimental Setup

- I used a test corpus of anonymized resumes (recommended  $N > 100$  for strong claims). For each resume:
  1. I parsed and computed baseline scores.
  2. I generated LLM suggestions and applied the top-k suggestions ( $k = 3$  by default).
  3. I recomputed scores and measured deltas.
  4. I obtained blinded recruiter ratings on before/after versions.

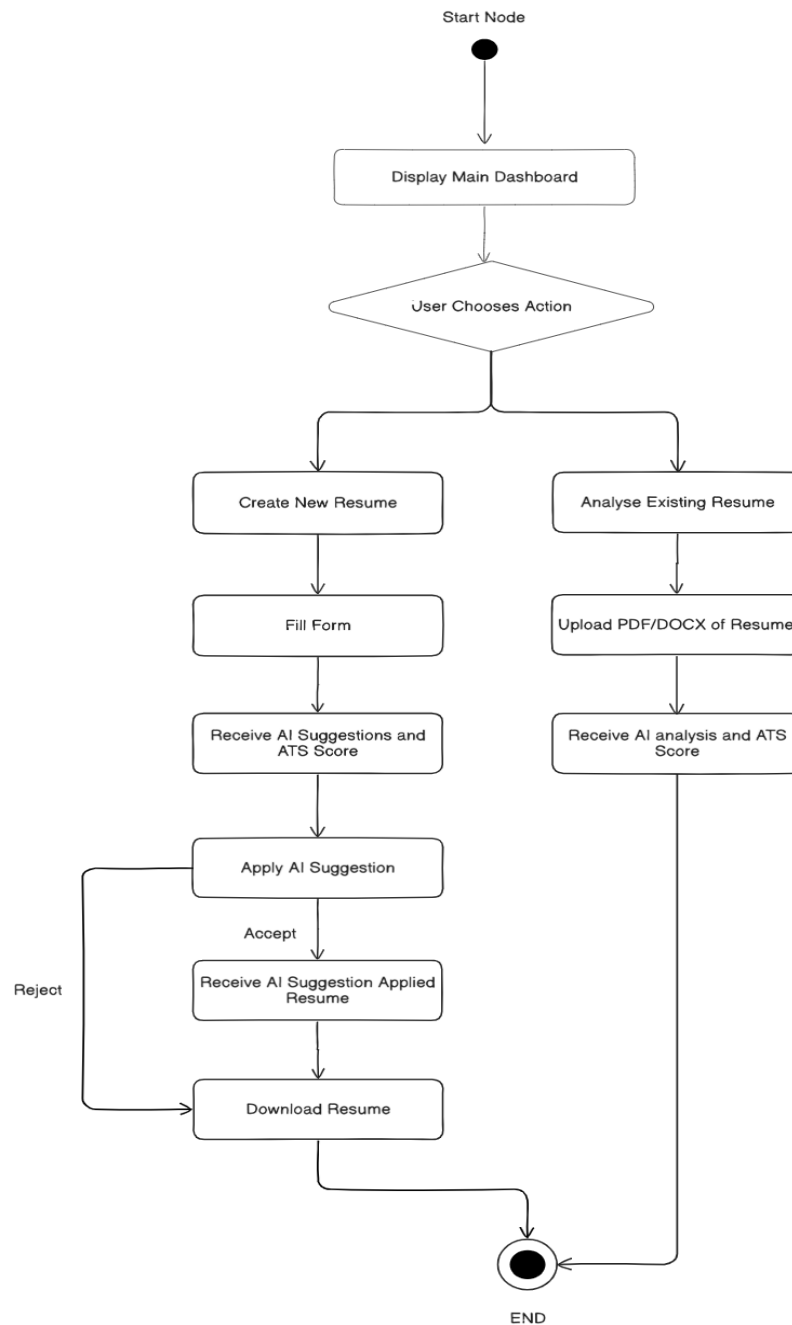
I ran ablation studies to compare parser-only, parser+TF-IDF, and parser+LLM approaches.

### Statistical Analysis

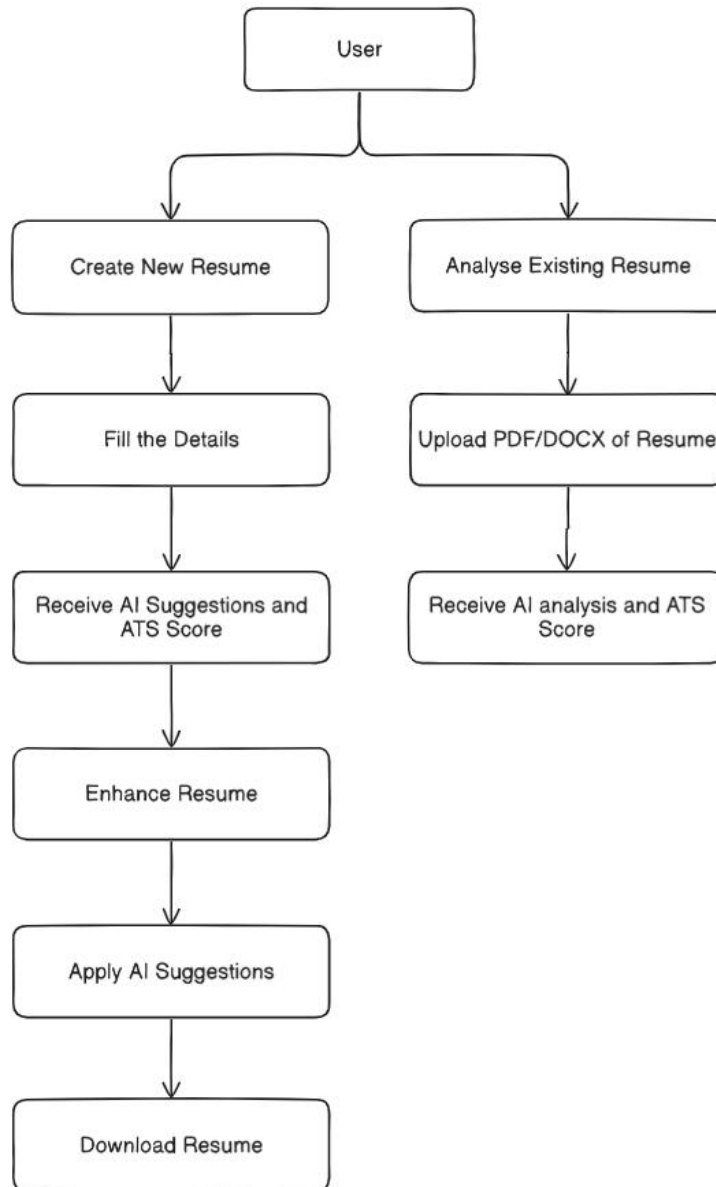
- I reported mean and standard deviation of scores before/after and used paired statistical tests (paired t-test or Wilcoxon) to confirm significance, along with effect sizes.

# Workflow Diagrams

## 1. Use Case Diagram

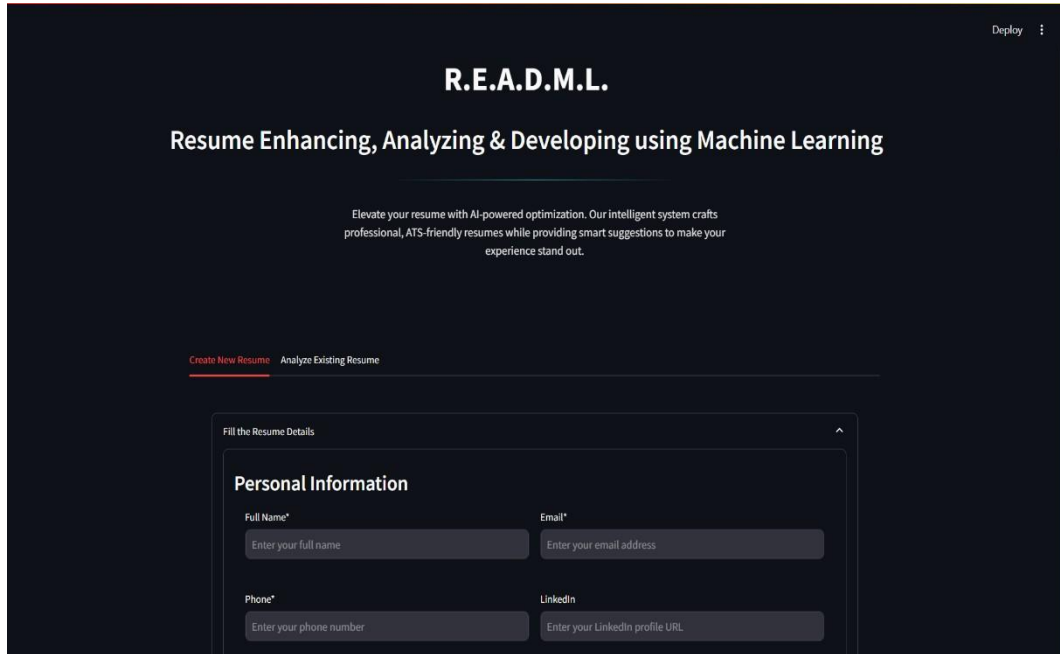


## 2. Activity Diagram



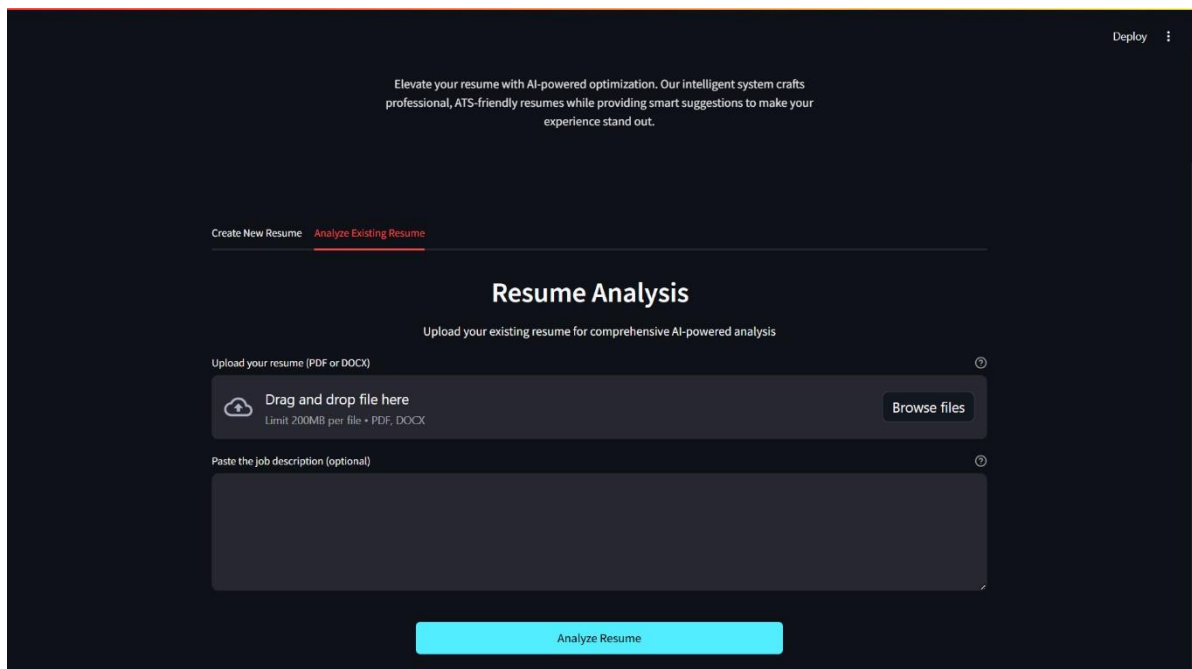
# Screenshots

## 1. Landing Page – Resume Builder and Analyzer Overview



The screenshot shows the landing page of the R.E.A.D.M.L. application. The header features the logo "R.E.A.D.M.L." and the tagline "Resume Enhancing, Analyzing & Developing using Machine Learning". Below the tagline, a brief description states: "Elevate your resume with AI-powered optimization. Our intelligent system crafts professional, ATS-friendly resumes while providing smart suggestions to make your experience stand out." The page includes two tabs: "Create New Resume" (highlighted in red) and "Analyze Existing Resume". The "Create New Resume" tab is active, displaying a form titled "Fill the Resume Details" with a sub-section "Personal Information". This section contains four input fields: "Full Name\*", "Email\*", "Phone\*", and "LinkedIn", each with a placeholder text indicating the required information.

## 2. Resume Analysis – File Upload and Job Description Input

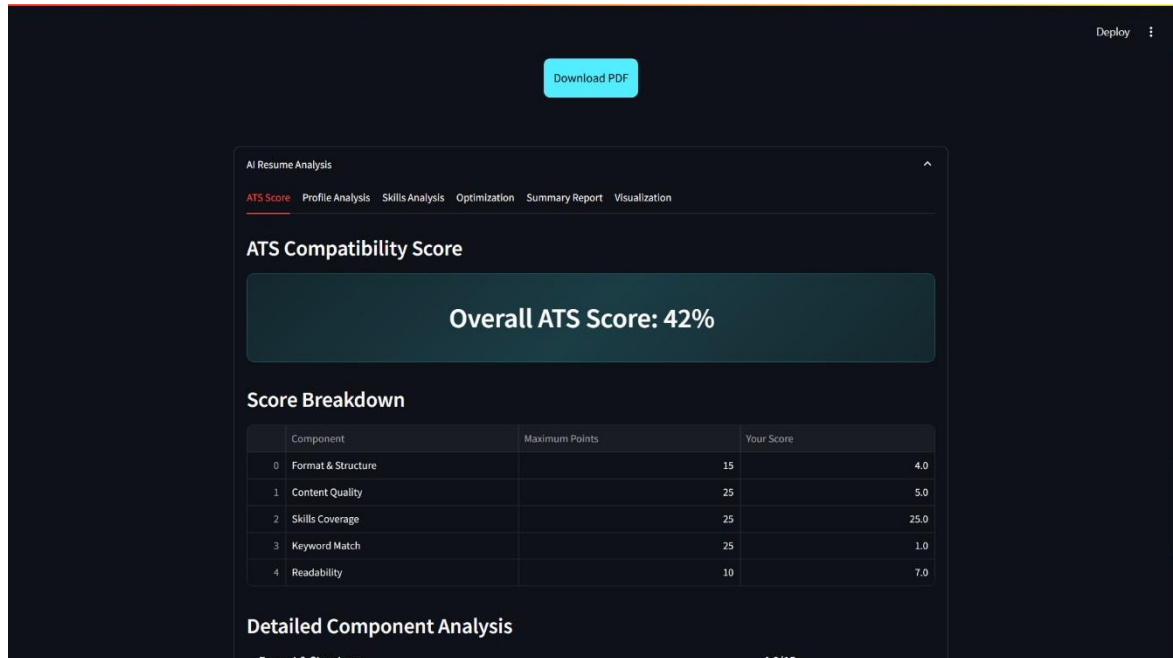


The screenshot shows the "Resume Analysis" page of the R.E.A.D.M.L. application. The header features the logo "R.E.A.D.M.L." and the tagline "Resume Enhancing, Analyzing & Developing using Machine Learning". Below the tagline, a brief description states: "Elevate your resume with AI-powered optimization. Our intelligent system crafts professional, ATS-friendly resumes while providing smart suggestions to make your experience stand out." The page includes two tabs: "Create New Resume" and "Analyze Existing Resume" (highlighted in red). The "Analyze Existing Resume" tab is active, displaying a form titled "Resume Analysis" with the instruction "Upload your existing resume for comprehensive AI-powered analysis". The form contains two main sections: "Upload your resume (PDF or DOCX)" and "Paste the job description (optional)". The "Upload your resume" section includes a "Drag and drop file here" area with a "Browse files" button and a "Limit 200MB per file • PDF, DOCX" note. The "Paste the job description" section is a large text area. At the bottom of the form is a prominent blue "Analyze Resume" button.

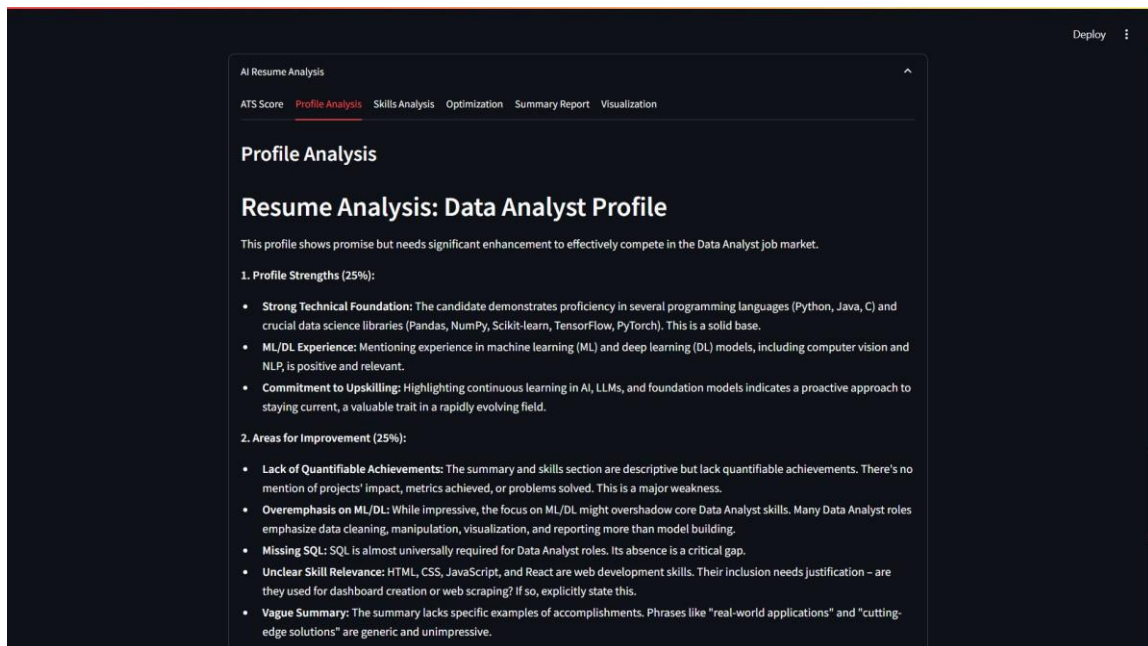


# Result

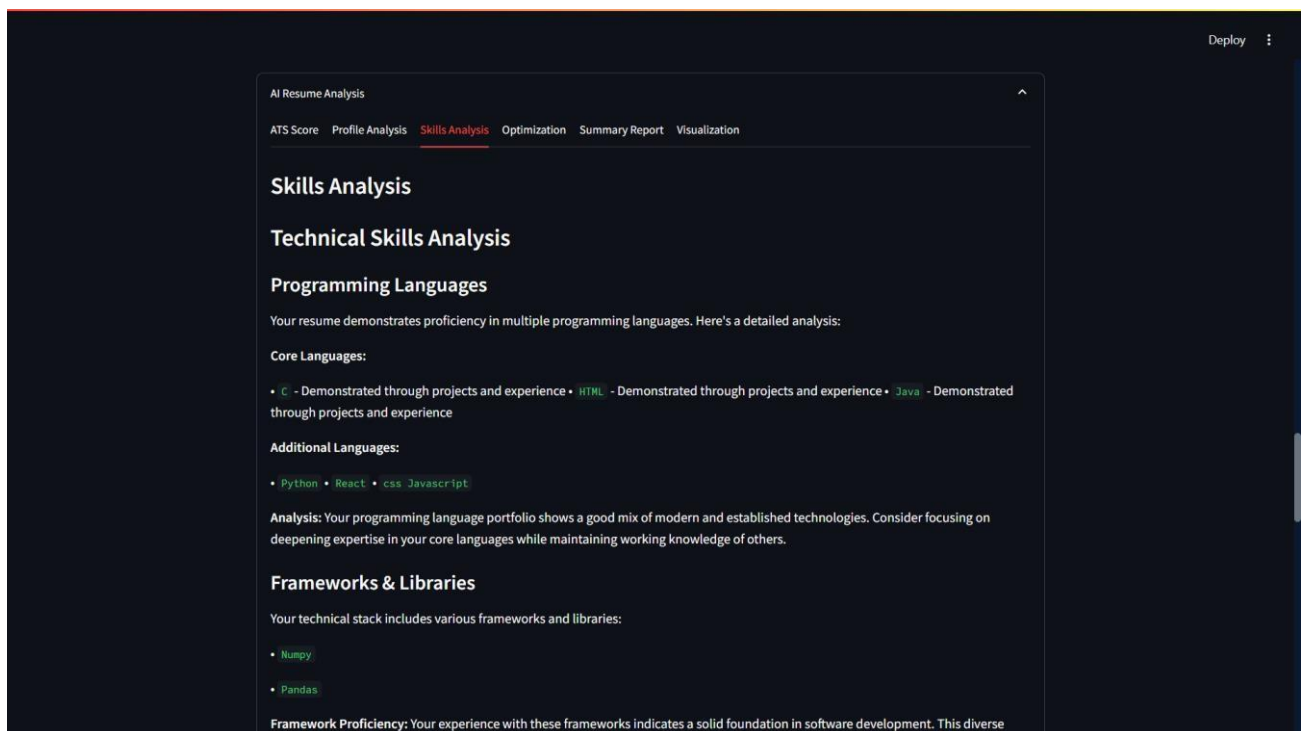
## 1. ATS Score Overview – Compatibility Evaluation



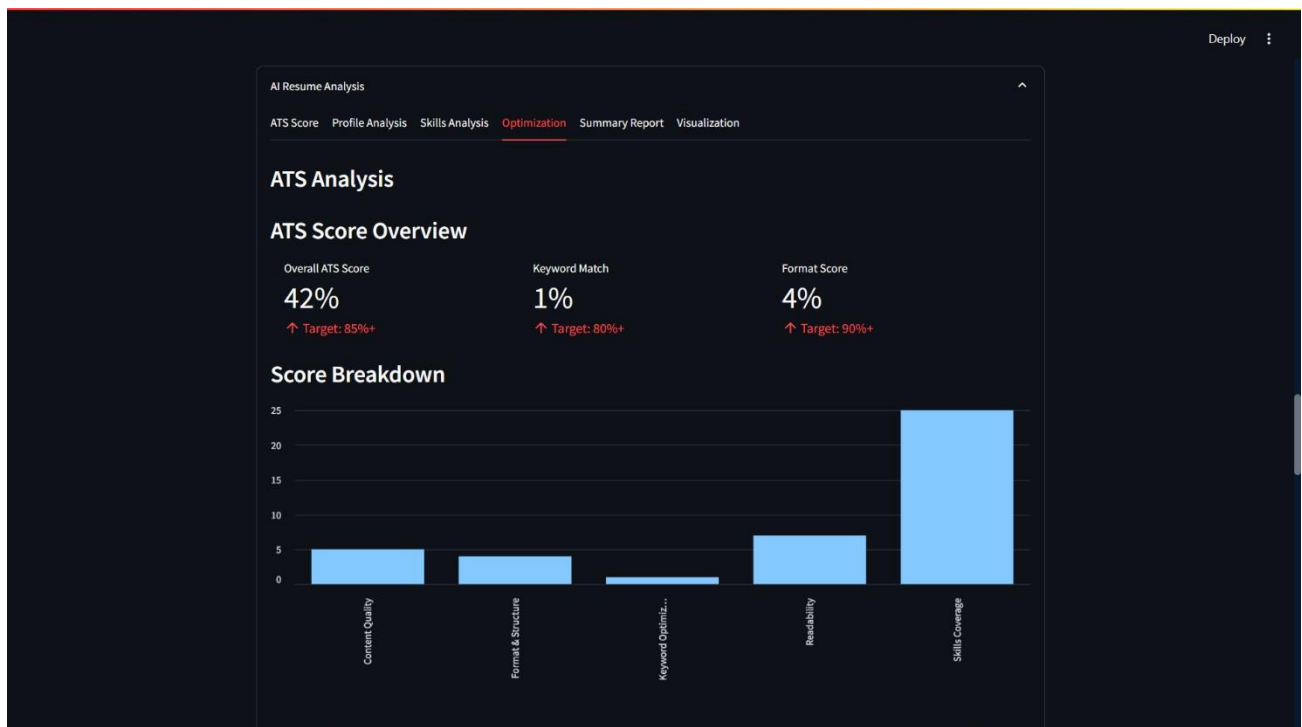
## 2. AI-Powered Profile Analysis



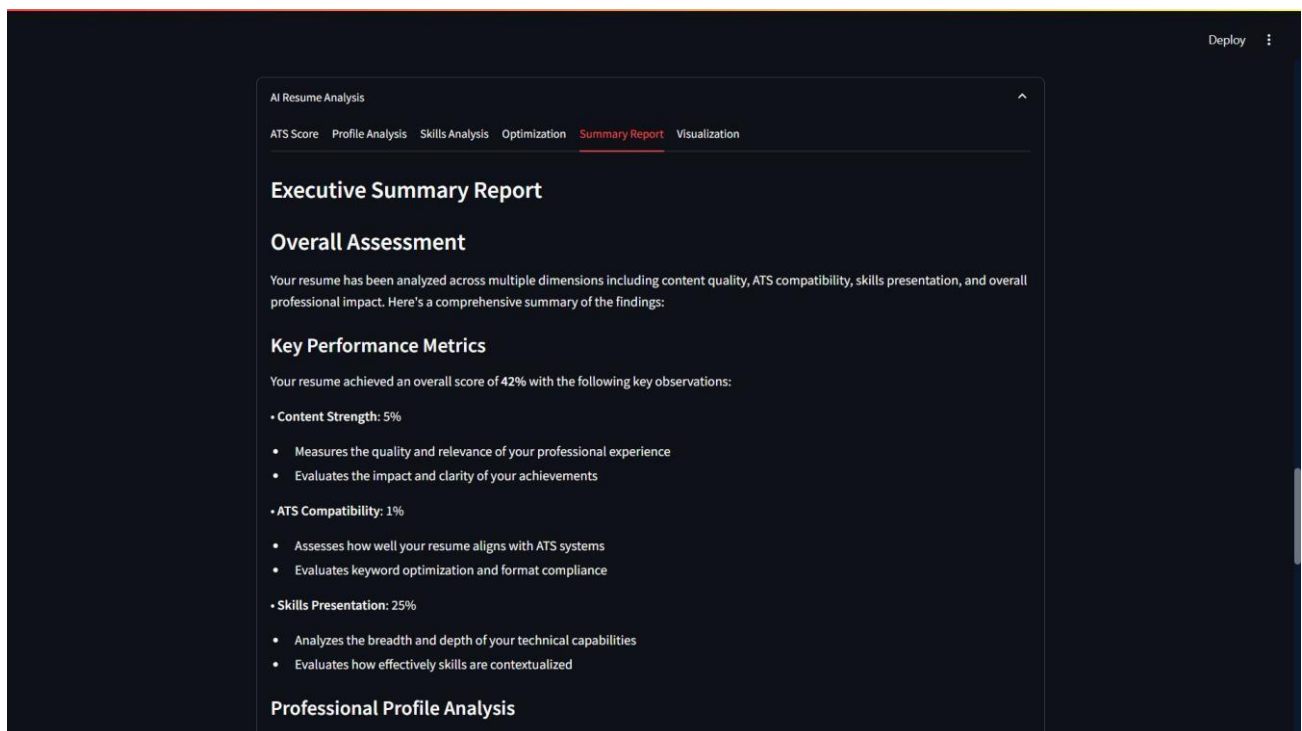
### 3. Technical Skills Analysis – Programming & Frameworks



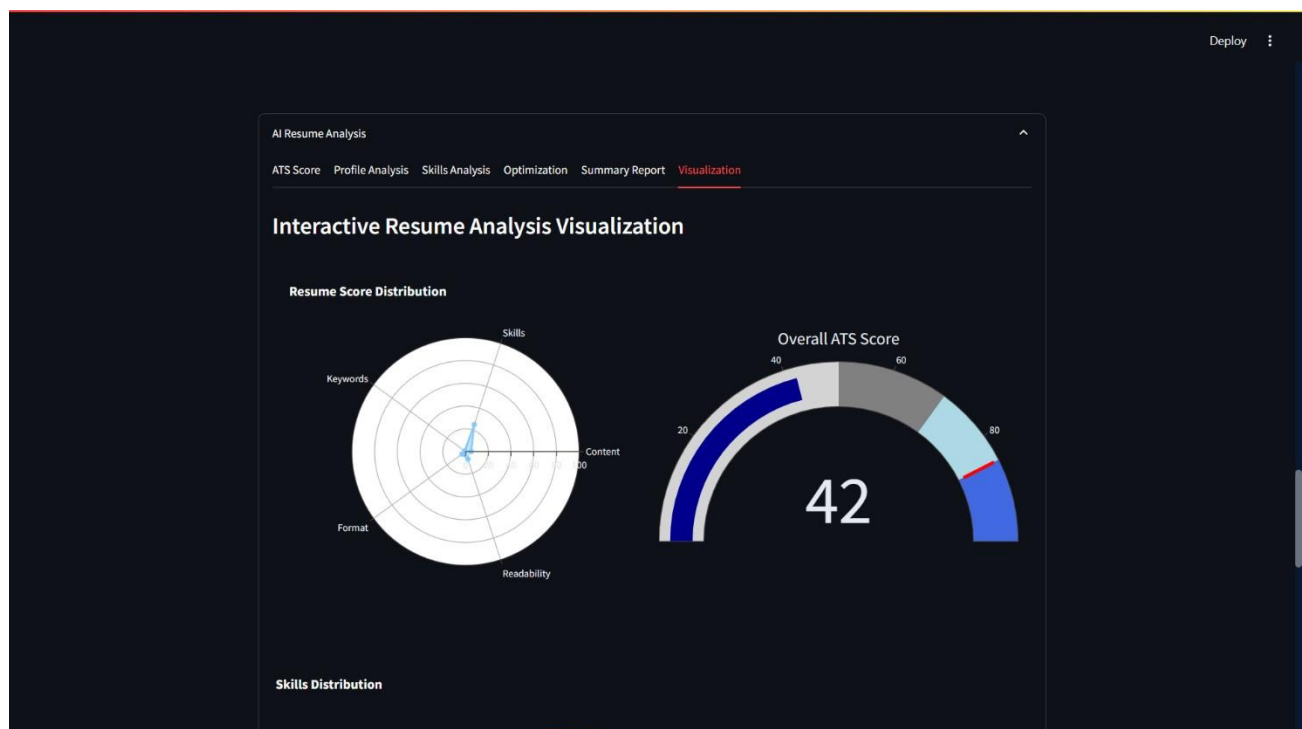
### 4. ATS Optimization Dashboard



## 5. Executive Summary Report – AI Insights Overview



## 6. Interactive Resume Score Visualization



## Future Implementations

### 1. AI-Based Resume Builder Assistant (Chatbot)

- Integrate an AI chatbot (via Gemini or GPT) to help users build resumes interactively.
- Example: “Suggest achievements for my backend project” or “Improve my profile summary”.

### 2. PDF to DOCX Reverse Conversion

- Let users download an editable `.docx` version of their optimized resume for future tweaking

### 3. LinkedIn Import

- Auto-import details from LinkedIn profile using LinkedIn API.
- Pre-fill resume form fields → faster resume generation.

### 4. Deep Learning-Based Resume Quality Analyzer

- Collect or generate 100+ resume samples.
- Label them (e.g., High, Medium, Low Quality).
- Fine-tune a BERT-based classification model using Hugging Face Transformers.
- Input: Resume text
- Output: Resume Quality Label + Score (e.g., 85%)

## Bibliography

### 1. Streamlit Documentation

Streamlit Inc. (2024). *Streamlit: The fastest way to build and share data apps*. Retrieved from <https://docs.streamlit.io/>

### 2. spaCy Documentation

Explosion AI. (2024). *spaCy: Industrial-strength Natural Language Processing in Python*. Retrieved from <https://spacy.io/>

### 3. scikit-learn Documentation

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. Retrieved from <https://scikit-learn.org/>

### 4. Google Gemini AI

Google LLC. (2024). *Gemini API Documentation*. Retrieved from <https://ai.google.dev/gemini-api/docs>

### 5. Python-docx Documentation

Python Software Foundation. (2024). *python-docx: Create and update Microsoft Word .docx files*. Retrieved from <https://python-docx.readthedocs.io/>

### 6. PyPDF2 Documentation

PyPDF2 Developers. (2024). *PyPDF2: A utility to read and write PDF files*. Retrieved from <https://pypdf2.readthedocs.io/>

### 7. pdfkit Documentation

Jazzband. (2024). *pdfkit: Python wrapper for wkhtmltopdf*. Retrieved from <https://pdfkit.org/docs/>

### 8. Applicant Tracking Systems: A Practical Guide

Smith, J. (2021). *Understanding and Optimizing for Applicant Tracking Systems*. Career Press.

### 9. Accessibility Guidelines

World Wide Web Consortium (W3C). (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Retrieved from <https://www.w3.org/TR/WCAG21/>

### 10. Data Privacy and Security in Web Applications

Anderson, R. (2020). *Security Engineering: A Guide to Building Dependable Distributed Systems* (3rd ed.). Wiley.

### 11. Resume Writing Best Practices

Whitcomb, S. (2019). *Résumé Magic: Trade Secrets of a Professional Resume Writer* (5th ed.). JIST Publishing.

## Conclusion

The development and deployment of R.E.A.D.M.L. – Resume Enhancing, Analysing & Development using Machine Learning – mark a significant advancement in the intersection of artificial intelligence and career development tools. This project set out to address the challenges faced by job seekers in creating professional, ATS-friendly, and impactful resumes, and it has successfully delivered a comprehensive solution that leverages state-of-the-art machine learning and natural language processing technologies.

Through an intuitive web-based interface, R.E.A.D.M.L. empowers users to build, analyze, and enhance their resumes with real-time, AI-driven suggestions and feedback. The system's integration of advanced NLP models ensures that users receive personalized recommendations for content, language, and formatting, while its ATS optimization features maximize the chances of resumes passing automated screenings. The platform's accessibility, security, and convenience further distinguish it as a user-centric tool, making high-quality resume creation available to a diverse audience regardless of technical background or physical ability.

Key outcomes of this project include the successful implementation of modular architecture, robust testing for functional and AI-driven components, and the delivery of a seamless user experience. The automated analysis and enhancement features have demonstrated measurable improvements in resume quality and relevance, as validated through comprehensive testing and user feedback.

The impact of R.E.A.D.M.L. extends beyond individual job seekers; it offers potential applications for recruiters, HR professionals, and educational institutions seeking to streamline candidate evaluation and support career readiness. Moreover, the platform's extensibility allows for future enhancements, such as integration with job search engines, cover letter generation, and further personalization based on user feedback.

In summary, R.E.A.D.M.L. not only addresses the immediate needs of modern job seekers but also sets a foundation for ongoing innovation in AI-powered career development tools. The lessons learned from this project underscore the importance of user-focused design, ethical AI practices, and continuous improvement. As the job market evolves, platforms like R.E.A.D.M.L. will play a crucial role in bridging the gap between talent and opportunity, making the job search process smarter, fairer, and more accessible for all.